

Exercise 2: Memory Matching Flutter Game

README — Configuration, Setup, and Project Documentation

GitHub Repository

The full source code for this project is publicly available on GitHub. You can browse the code, download the project, or clone it directly using the link below.

Repository: omavaiya4/memory_match

https://github.com/omavaiya4/memory_match/tree/master

Branch: master

To clone the repository locally, open a terminal and run:

```
git clone https://github.com/omavaiya4/memory_match.git
```

1. Overview

This document describes the Memory Match application developed for Exercise 2 of CS5450 Mobile Programming. The app is a classic card-flipping memory game built entirely using Flutter and Dart, without any third-party game libraries. The player selects a difficulty level, then taps cards to flip them and find matching emoji pairs. The game tracks the number of moves made, pairs found, and total time elapsed. When all pairs are matched, a congratulatory dialog appears summarising the player's performance.

The application was built and tested using Android Studio on a real Android device. It runs on Android (API 21 and above), and can also be launched on Chrome or Windows desktop through Flutter's multi-platform support.

2. Project Structure

Below is the complete folder and file layout of the project as it appears when extracted from the ZIP file and opened in Android Studio.

```
memory_match/ <- Project root (open this in Android Studio)
lib/ <- All Dart source files
main.dart <- Entry point; MaterialApp + HomeScreen
```

```
game_screen.dart <- Main game UI (grid, stats bar, restart)
game_controller.dart <- All game logic, timer, match detection
flip_card_widget.dart <- 3D flip animation widget
card_model.dart <- Data class: emoji, isFlipped, isMatched
android/ <- Android platform files
app/
build.gradle <- App-level Gradle config
src/main/
AndroidManifest.xml
kotlin/.../MainActivity.kt
res/ <- Drawables, styles
build.gradle <- Project-level Gradle config
settings.gradle
gradle.properties
gradle/wrapper/
gradle-wrapper.properties <- Gradle version (set to 8.9)
test/
widget_test.dart <- Basic smoke test
pubspec.yaml <- Flutter dependencies
analysis_options.yaml <- Lint rules
README.md <- GitHub README
README.pdf <- This document
```

Key Source Files

File	Purpose
main.dart	Launches the app. Contains MaterialApp setup and the HomeScreen widget with three difficulty buttons (Easy, Medium, Hard).
game_screen.dart	The main game view. Displays the stats bar (moves, pairs, timer), the card grid, and the restart button at the bottom.
game_controller.dart	Handles all game logic: shuffling cards, tracking which cards are flipped, detecting matches, running the timer, and notifying the UI via ChangeNotifier.
flip_card_widget.dart	A stateful widget that animates a 3D card flip using AnimationController and Matrix4 perspective transform. No external animation libraries needed.
card_model.dart	A lightweight data class representing one card: its emoji symbol, whether it is currently flipped face-up, and whether it has been matched.

3. Prerequisites

Before running the project, make sure the following tools are installed on your machine.

- **Flutter SDK 3.10 or higher.** Download from <https://flutter.dev/docs/get-started/install> and add the bin folder to your system PATH.
- **Android Studio (Hedgehog 2023.1.1 or newer).** Includes the Android SDK, an AVD emulator, and a bundled JDK (Java 21). Download from <https://developer.android.com/studio>.
- **Flutter and Dart Plugins for Android Studio.** Install via File > Settings > Plugins. Search for Flutter and install it; the Dart plugin installs automatically.
- **Android SDK API 21 or higher.** Installed automatically with Android Studio. API 33 or 34 is recommended for the emulator system image.
- **Java 21 (bundled with Android Studio).** Flutter uses the JDK bundled inside Android Studio automatically. No separate Java installation is needed.

4. How to Configure and Run the App

Step 1 — Clone or Extract the Project

Either clone the GitHub repository using the command below, or extract the submitted ZIP file to a folder on your computer. Both will give you a `memory_match` folder containing `pubspec.yaml` at its root.

```
git clone https://github.com/omavaiya4/memory_match.git
```

Step 2 — Open the Project in Android Studio

Open Android Studio and go to File > Open. Navigate to the `memory_match` folder (the one that contains `pubspec.yaml`) and click OK. Android Studio will recognise it as a Flutter project and begin indexing the files.

Step 3 — Install Flutter and Dart Plugins (first time only)

Go to File > Settings > Plugins and search for Flutter. Install it — the Dart plugin is installed automatically at the same time. Restart Android Studio when prompted.

Step 4 — Get Flutter Dependencies

Open the Terminal tab at the bottom of Android Studio and run the following command. This downloads all packages listed in `pubspec.yaml`.

```
flutter pub get
```

Step 5 — Fix the Gradle Version (Required)

Flutter uses Java 21 internally, which requires Gradle 8.5 or higher. Open the file `android/gradle/wrapper/gradle-wrapper.properties` and make sure the `distributionUrl` line reads exactly as shown below. If it says `gradle-8.0-all`, change it to `gradle-8.9-all` and save the file.

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.9-all.zip
```

Step 6 — Create and Start an Android Emulator

Go to Tools > Device Manager and click Create Device. Choose a phone model such as Pixel 6, select a system image (API 33 or 34 recommended), and click Finish. Then press the Play button next to the device to start the emulator.

Step 7 — Run the Application

Select your running emulator from the device dropdown at the top of Android Studio, then click the green Run button or press Shift + F10. You can also run the command below from the terminal in the project root folder. The first build takes approximately one to three minutes.

```
flutter run
```

5. App Features and Game Logic

- Three difficulty levels from the home screen: Easy (4x3 grid, 6 pairs), Medium (4x4 grid, 8 pairs), and Hard (4x5 grid, 10 pairs).
- Smooth 3D card flip animation using Flutter's AnimationController and Matrix4 perspective transform. No external animation libraries are used.
- Each game session shuffles a unique set of emoji pairs so the card layout is different every time.
- Six rotating background colours on card backs make the grid visually engaging without requiring image assets.
- A live stats bar tracks moves made, pairs found out of the total, and elapsed time in MM:SS format.
- Unmatched card pairs automatically flip back after a short delay. Matched pairs remain face-up with a green border.
- A win dialog appears when all pairs are found, showing total time and move count with options to replay or return to the home menu.
- A Restart button on the game screen lets the player reset the board at any time without going back to the home screen.
- Card dimensions are calculated dynamically so the layout adapts to different screen sizes and orientations.
- The app uses a simple ChangeNotifier pattern: GameController holds all state and calls notifyListeners(), while GameScreen rebuilds in response.

6. App Screenshots

The following screenshots are taken directly from the running app on an Android device, demonstrating the Home Screen and two difficulty levels of the Game Screen.

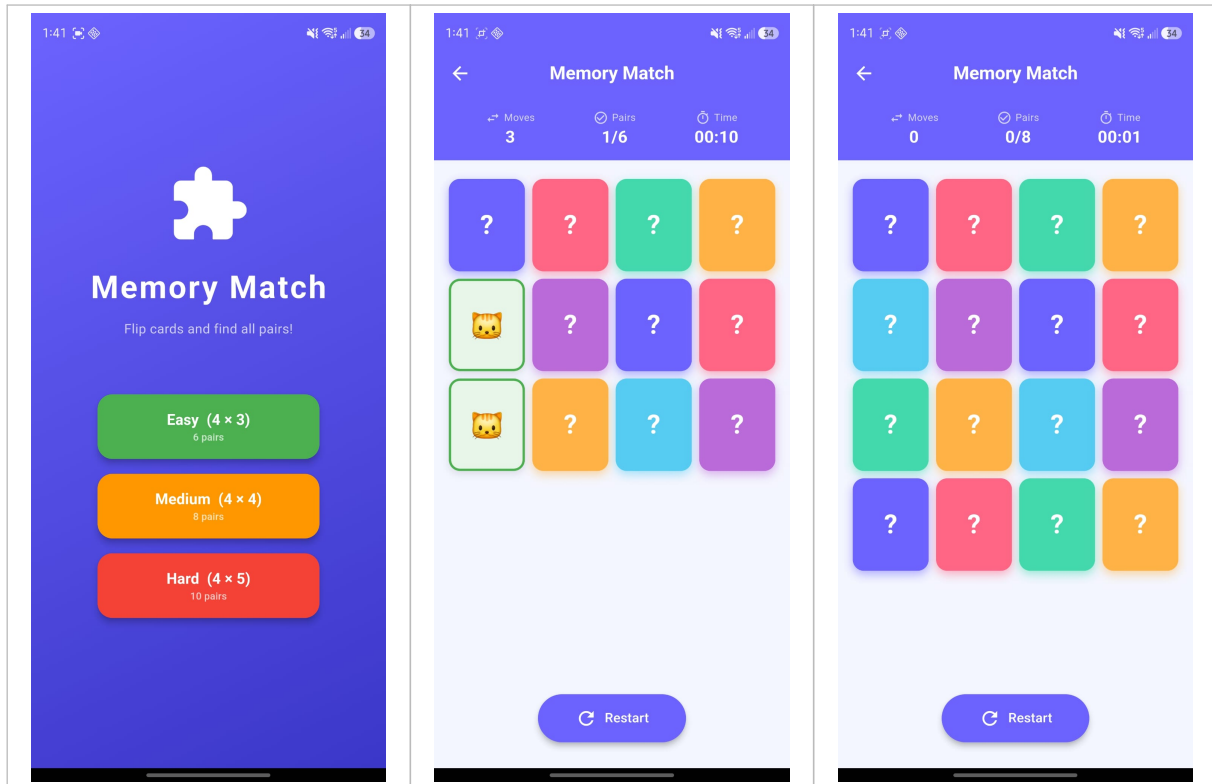


Figure 1 — Home Screen Difficulty selection with Easy, Medium, and Hard buttons.

Figure 2 — Easy Mode (4x3) One pair matched (cat emoji). Stats bar shows Moves, Pairs, Time.

Figure 3 — Medium Mode (4x4) All 16 cards face-down at the start of a new game.

7. Common Issues and Fixes

- **Problem:** Gradle version incompatible with Java

Fix: Open `android/gradle/wrapper/gradle-wrapper.properties` and change the Gradle version to 8.9 as described in Step 5. Flutter uses Java 21 internally which requires Gradle 8.5 or higher.

- **Problem:** flutter: command not found

Fix: Flutter's bin directory is not on the system PATH. Add it (e.g. `C:\flutter\bin` on Windows) and restart your terminal or Android Studio.

- **Problem:** flutter pub get fails

Fix: Check your internet connection and confirm that pubspec.yaml is in the root of the folder opened in Android Studio.

- **Problem:** Emulator not appearing in the device dropdown

Fix: Open Tools > Device Manager and start the emulator manually by pressing its Play button, then click the dropdown again.

- **Problem:** App crashes immediately on launch

Fix: Run flutter clean followed by flutter pub get in the terminal, then try flutter run again.

- **Problem:** Plugin not found error during Gradle build

Fix: The project uses the legacy apply-from method to load the Flutter Gradle plugin. Make sure Flutter is installed and that flutter pub get has been run at least once.